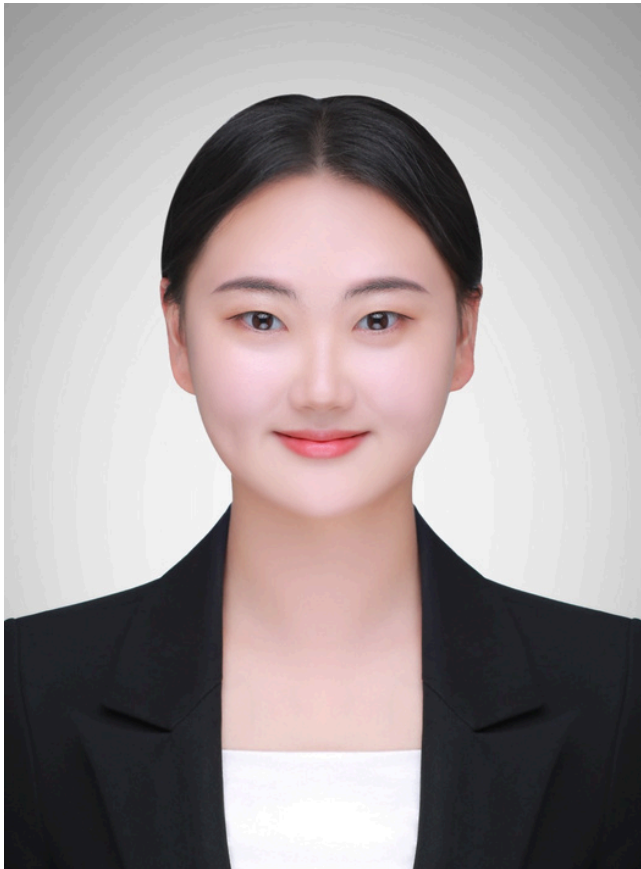

PORTFOLIO

문제를 스스로 정의하고 해결하는 개발자 표현경

girls0330@naver.com

010-6798-0076



Frontend Developer
표현경

교육

패스트캠퍼스 / 프론트엔드 3기
(24.12 ~ 25.07)
eBrainsoft / 웹개발 스터디 11기
(24.07 ~ 24.09)
현업 풀스택 개발자 1:1 멘토링
(23.12 ~ 24.05)

학력

학점은행제 (2025.03 ~ 진행중)
• 주 전공: 컴퓨터공학

호남대학교 (2018.03 ~ 2020.02)
• 주 전공: 중국어학과
• 이수학기: 4학기 (2학년 수료)
• 학점: 3.1 / 4.5
• 비고: 자퇴

스킬 / 툴

프로그래밍 언어	HTML5, CSS, React, TypeScript, JavaScript(ES6+),Java
템플릿/뷰 기술	React, Spring Boot, jQuery
버전 관리 & 협업	Git, GitHub, Slack
디자인	Figma

목차

01	AI 부동산 추천_줍줍 프로젝트 (25.05~25.07)		(3p ~ 6p)
02	동영상 공유_IdolLink 프로젝트 (25.03~25.04)		(7p ~ 10p)

AI 부동산 추천_줍줍 프로젝트 (25.05~25.07)

나만을 위한 AI 매물 추천 플랫폼

ZOOP

구성 인원

프론트엔드 4명, 백엔드 7명, UXUI디자이너 2명

배경

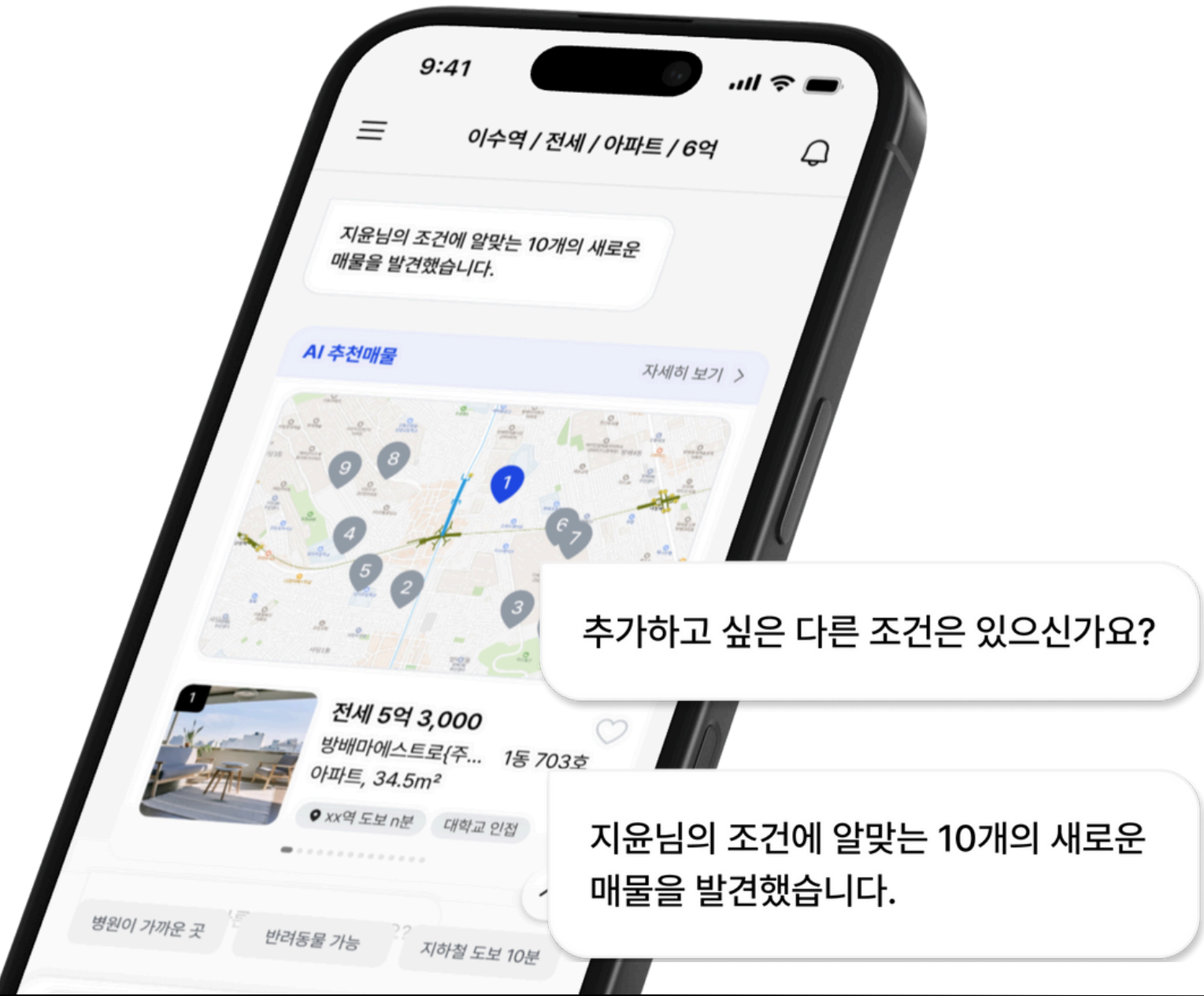
기존 부동산 플랫폼의 복잡한 필터링 과정과 전세 사기 증가 문제에 대응하여, AI 추천 시스템으로 개인 맞춤형 매물을 신속하게 제공하고 실제 거주자 리뷰와 커뮤니티를 통해 검증된 신뢰할 수 있는 부동산 서비스가 필요하다고 판단 되어 기업 협업 프로젝트를 참여

목표

AI 추천 시스템의 데이터를 직관적이고 효율적으로 표현하여 클릭 수를 최소화하고, 빠른 전환과 매끄러운 사용자 경험을 제공

기술 스택

Next.js, TypeScript, Zustand, TanStack Query, Tailwind CSS, shadcn/ui



AI 부동산 추천_줍줍 프로젝트 (25.05~25.07)

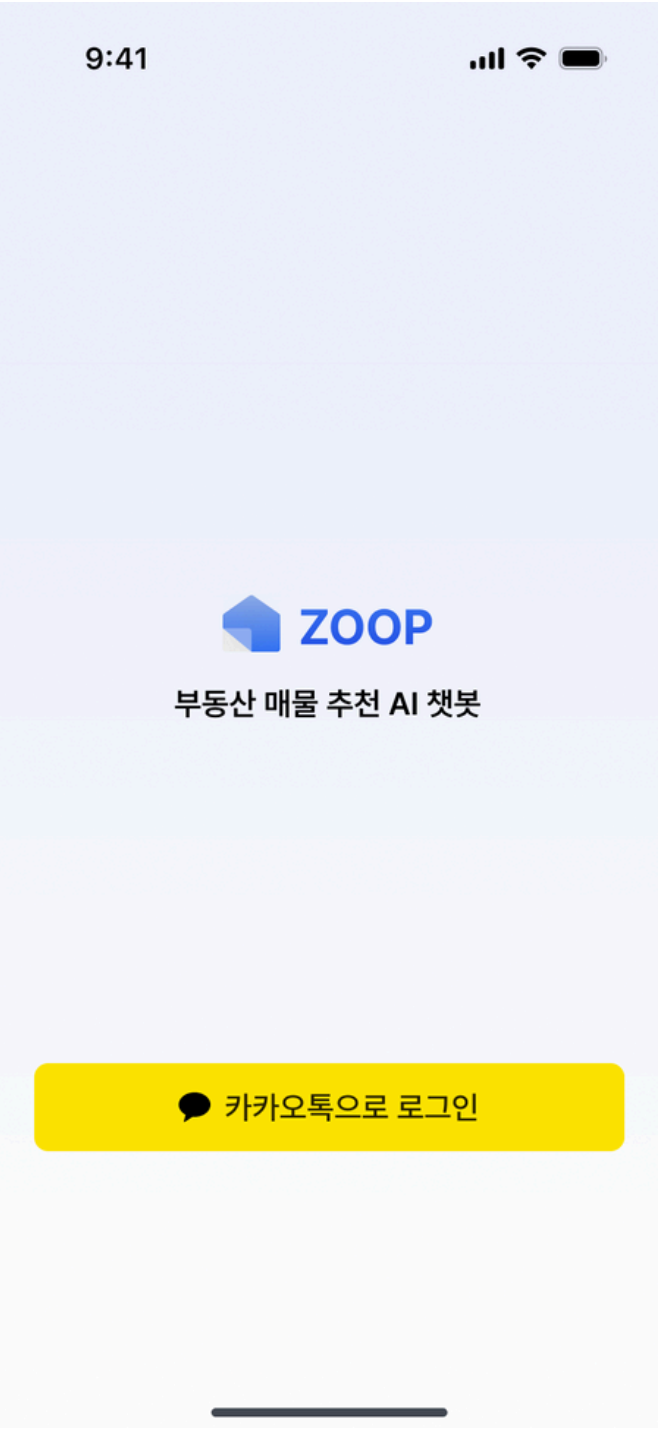
[로그인 페이지]

▶ 주요 작업

- 쿠키에 담긴 Token 관리
- 로그인한 유저 정보를 zustand를 사용해 전역적으로 관리

▶ 고민 지점

- 백엔드에서 발급한 인증 쿠키가 프론트엔드로 전달되지 않아 Next.js Middleware에서 인증 상태를 확인할 수 없는 문제가 발생했습니다. 해당 이슈를 비교적 늦게 발견했고, 프로젝트 일정상 서버 설정을 수정하기 어려운 상황이었습니다. 단기간 내 서비스 이용에 문제가 없도록 클라이언트 측에서 useAuthGuard hooks 구현해 사용자 정보 API 호출로 인증을 검증하고, 미인증 사용자를 로그인 페이지로 리다이렉트하는 방식으로 우선 대응했습니다.
- 프로젝트 종료 후 원인을 분석한 결과, 백엔드와 프론트엔드가 서로 다른 origin에서 동작하면서 브라우저의 쿠키 정책(SameSite, credentials)에 의해 쿠키가 차단되는 구조적 문제임을 확인했습니다. 향후에는 동일 origin 환경을 구성하거나 BFF(Backend for Frontend) 패턴을 도입해 reverse proxy 방식으로 쿠키 스코프를 통일함으로써 근본적으로 해결할 수 있을 것으로 판단했습니다.



< 로그인 페이지 >



< 닉네임 설정 페이지 >

AI 부동산 추천_줍줍 프로젝트 (25.05~25.07)

[매물 리스트 페이지]

▶ 주요 작업

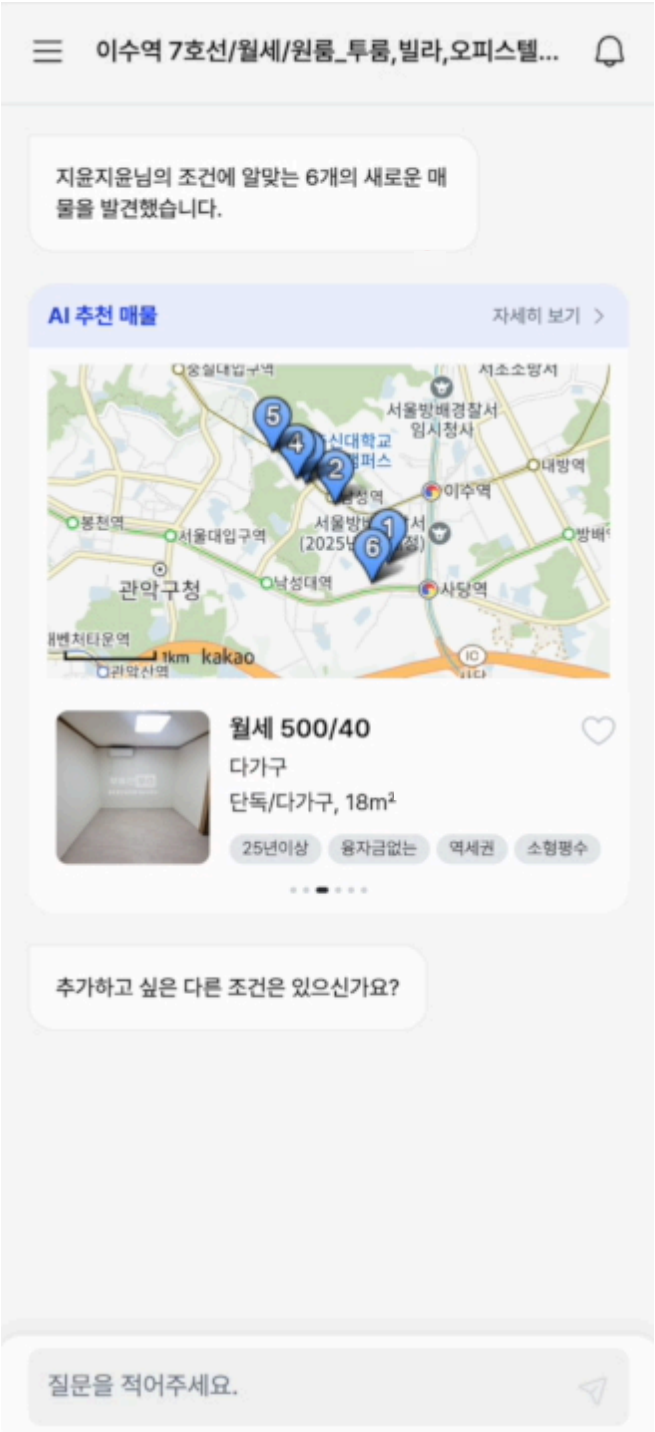
- Dialog 컴포넌트 구현 / 적용
- 카카오 맵 API를 적용
- 커스텀 마커 구현 (정렬 변경 시 리스트 순서에 맞게 업데이트)

▶ 고민 지점

- 지도 페이지와 매물 상세 페이지를 일반적인 페이지 이동 방식으로 구성하면, 사용자가 매물을 확인할 때마다 지도 맥락이 끊기고 재로딩되는 문제가 있었습니다. 특히 여러 매물을 비교하며 탐색하는 과정에서 반복적인 화면 전환으로 인해 탐색 흐름이 자주 끊길 것으로 예상했습니다.

이를 해결하기 위해 지도를 별도 페이지로 분리하지 않고 레이어 기반 구조로 설계했습니다. Dialog 컴포넌트를 활용한 모달 방식을 적용하여 페이지 이동 없이 지도 위에서 매물 정보를 바로 확인할 수 있도록 구현했습니다.

그 결과 사용자는 지도 컨텍스트를 유지한 채로 빠르게 매물을 탐색할 수 있었고, 화면 전환으로 인한 인지 부담과 로딩 지연을 줄여 더 매끄러운 탐색 경험을 제공할 수 있었습니다.



< 채팅 페이지 >



< 매물 목록 페이지 >

AI 부동산 추천_줍줍 프로젝트 (25.05~25.07)

[공통 컴포넌트]

▶ 주요 작업

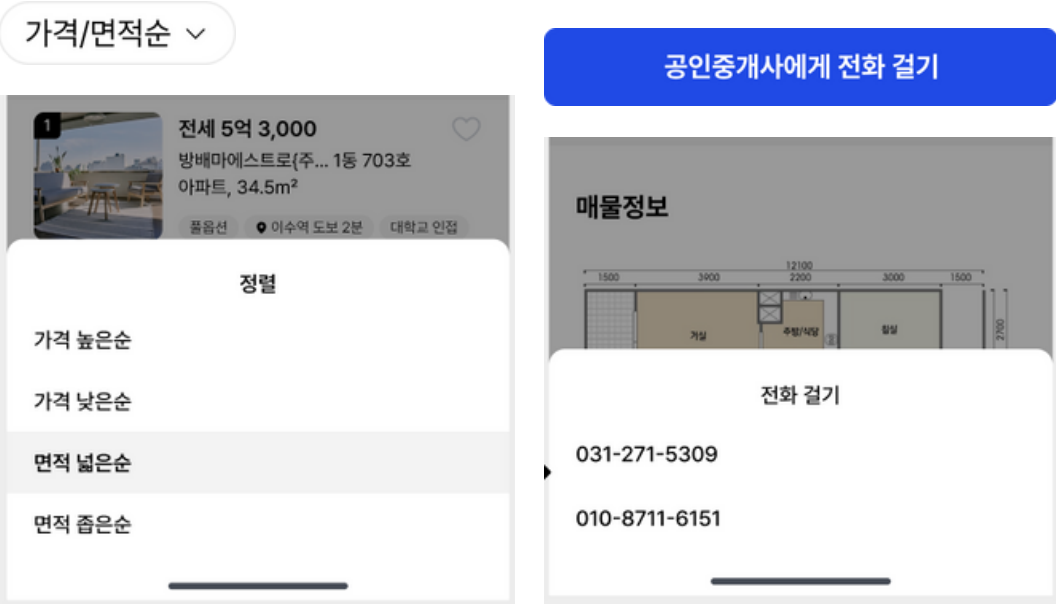
- shadcn/UI 기반 공통 UI 컴포넌트
Input, Textarea, Dropdown, BottomSheet, Dialog 개발

▶ 고민 지점

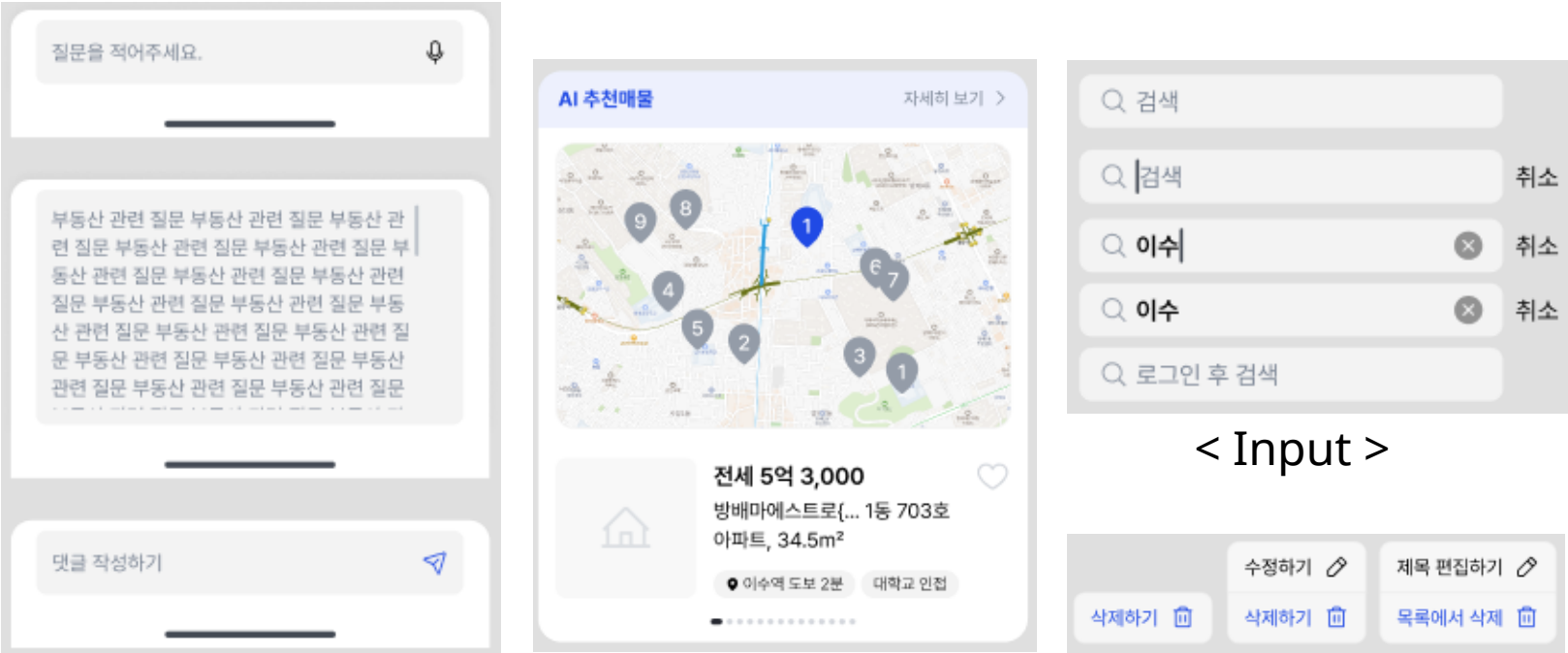
- BottomSheet 컴포넌트를 처음 구현할 때 trigger 버튼을 내부에 고정된 UI로 제공했는데, 화면마다 다른 디자인 요구사항이 생기면서 재사용이 어려워졌습니다. 버튼 스타일을 수정할 때마다 컴포넌트 내부를 분기하거나 props가 계속 늘어나면서 유지보수성이 떨어지는 문제가 있었습니다.

이를 해결하기 위해 BottomSheet가 버튼 UI를 직접 가지지 않고, children 함수 패턴을 적용해 부모 컴포넌트에서 trigger를 자유롭게 렌더링하도록 구조를 변경했습니다.

그 결과 BottomSheet는 열고 닫는 "동작"만 책임지고 UI는 외부에서 제어하는 구조가 되었으며, 다양한 디자인 요구사항을 수용하면서도 재사용성과 확장성을 함께 확보할 수 있었습니다.



< Bottom Sheet >



< Textarea >

< MapViewer >

< Dropbox >

동영상 공유_IdolLink 프로젝트 (25.03~25.04)

https://idollink.netlify.app

구성 인원

프론트엔드 3명

배경

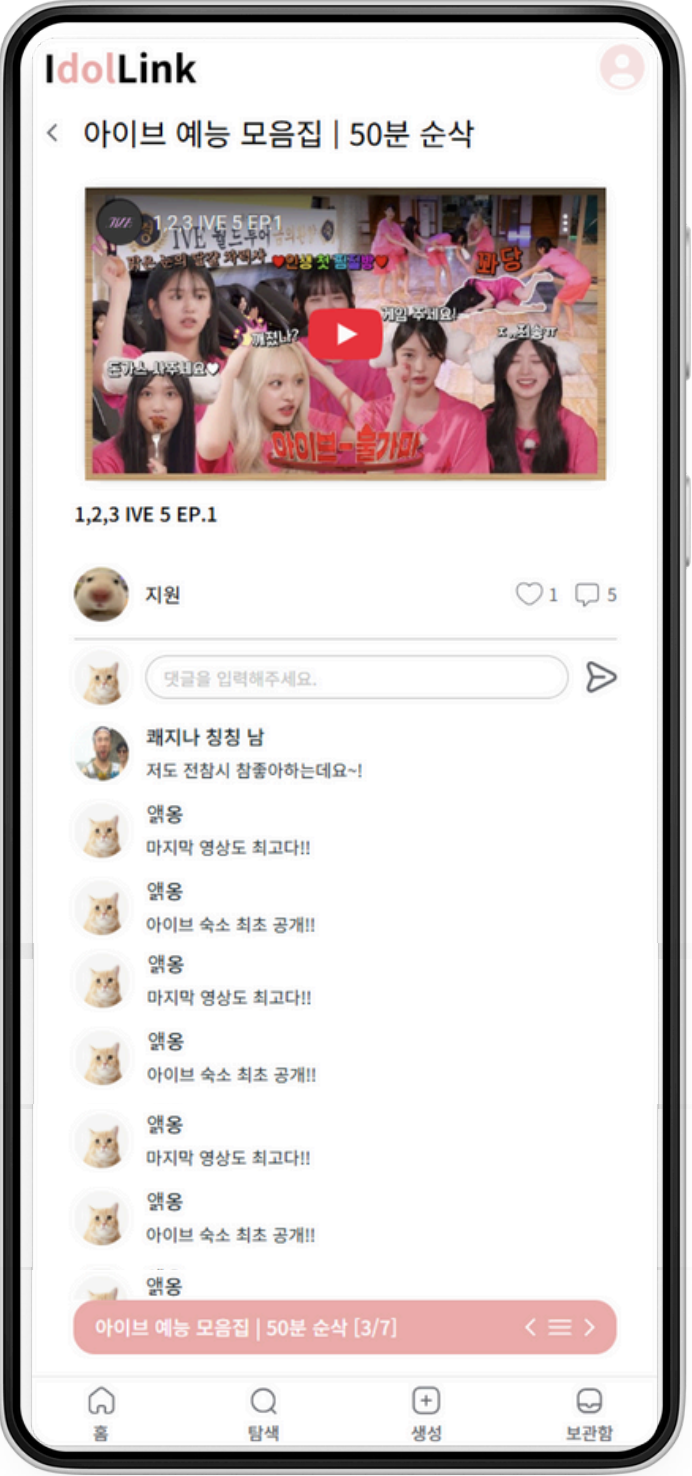
유튜브 플랫폼엔 영상은 많지만 아이돌 팬들을 위한 커뮤니티 기능이 부족하다 느껴, 같은 아이돌을 좋아하는 팬들끼리 취향을 공유하고 소통할 수 있는 전용 플랫폼을 만들고 싶었음

목표

아이돌 팬들을 위한 영상 중심의 소셜 플랫폼을 만들어, 개인의 취향 표현과 팬 커뮤니티 형성을 동시에 지원하는 서비스 개발

기술 스택

React, TypeScript, Vite, Zustand, TanStack Query, Emotion, Supabase



동영상 공유_IdolLink 프로젝트 (25.03~25.04)

[서버/DB]

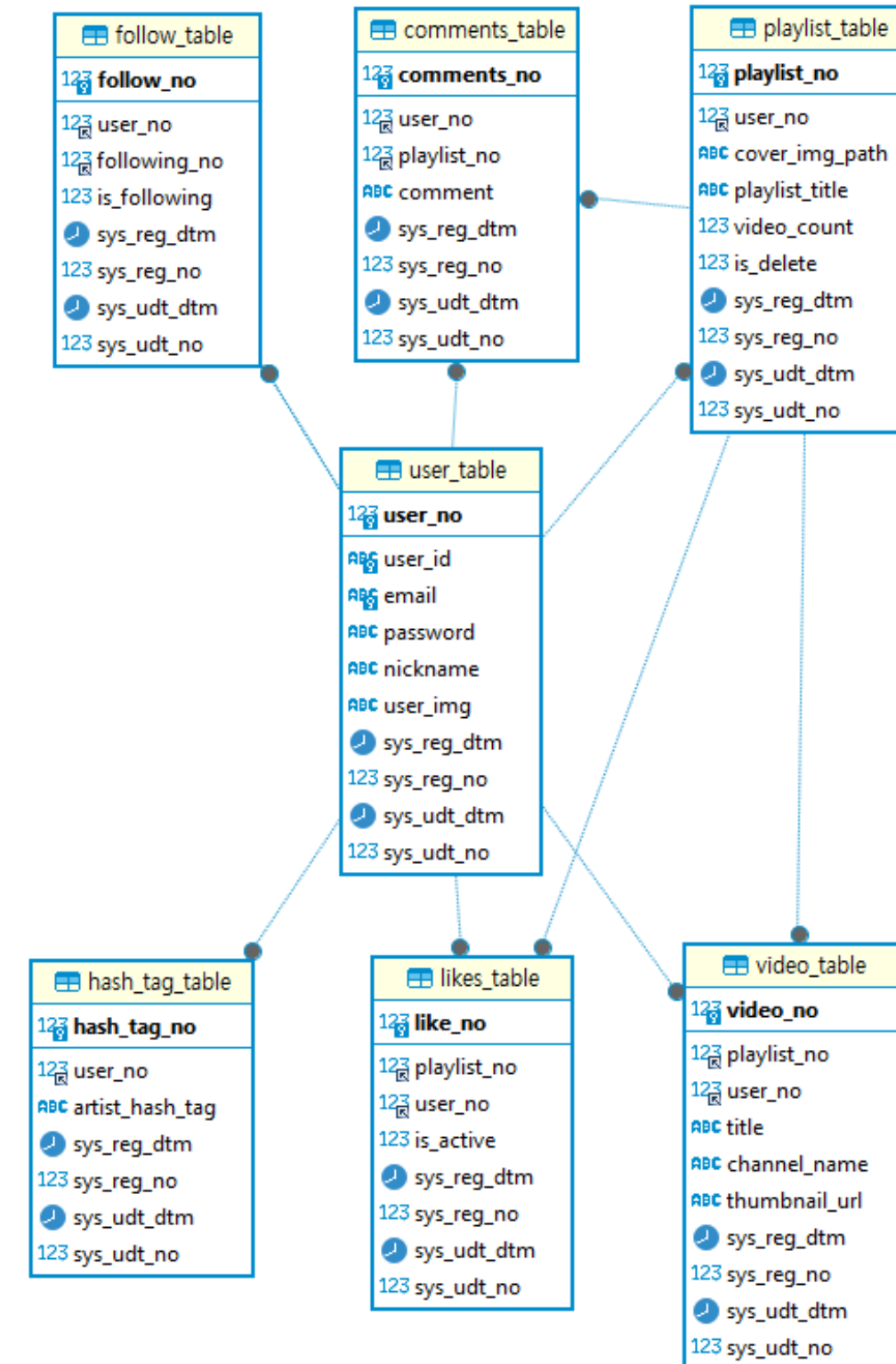
▶ 주요 작업

- Supabase 기반의 관계형 DB 설계 및 구현

▶ 고민 지점

- Supabase 도입 초기, 테스트 중 인증 없이도 다른 사용자의 게시글과 개인 정보에 접근 가능한 보안 취약점을 발견했습니다. 기존 Java 백엔드에서는 서버 계층에서 모든 요청을 검증했지만, Supabase는 클라이언트가 DB에 직접 접근하므로 데이터베이스 레벨에서 권한을 제어해야 한다는 점을 깨달았습니다.

이를 해결하기 위해 Row Level Security(RLS)를 도입하여, 사용자는 `auth.uid() = user_id` 조건으로 자신의 데이터만 조회/수정 가능하도록 정책을 설계했습니다. 이 과정을 통해 전통적인 서버 중심 보안과 데이터베이스 레벨 보안의 차이를 이해하고, BaaS 아키텍처에서의 보안 설계 방식을 체득할 수 있었습니다.



동영상 공유_IdolLink 프로젝트 (25.03~25.04)

[홈/탐색 페이지]

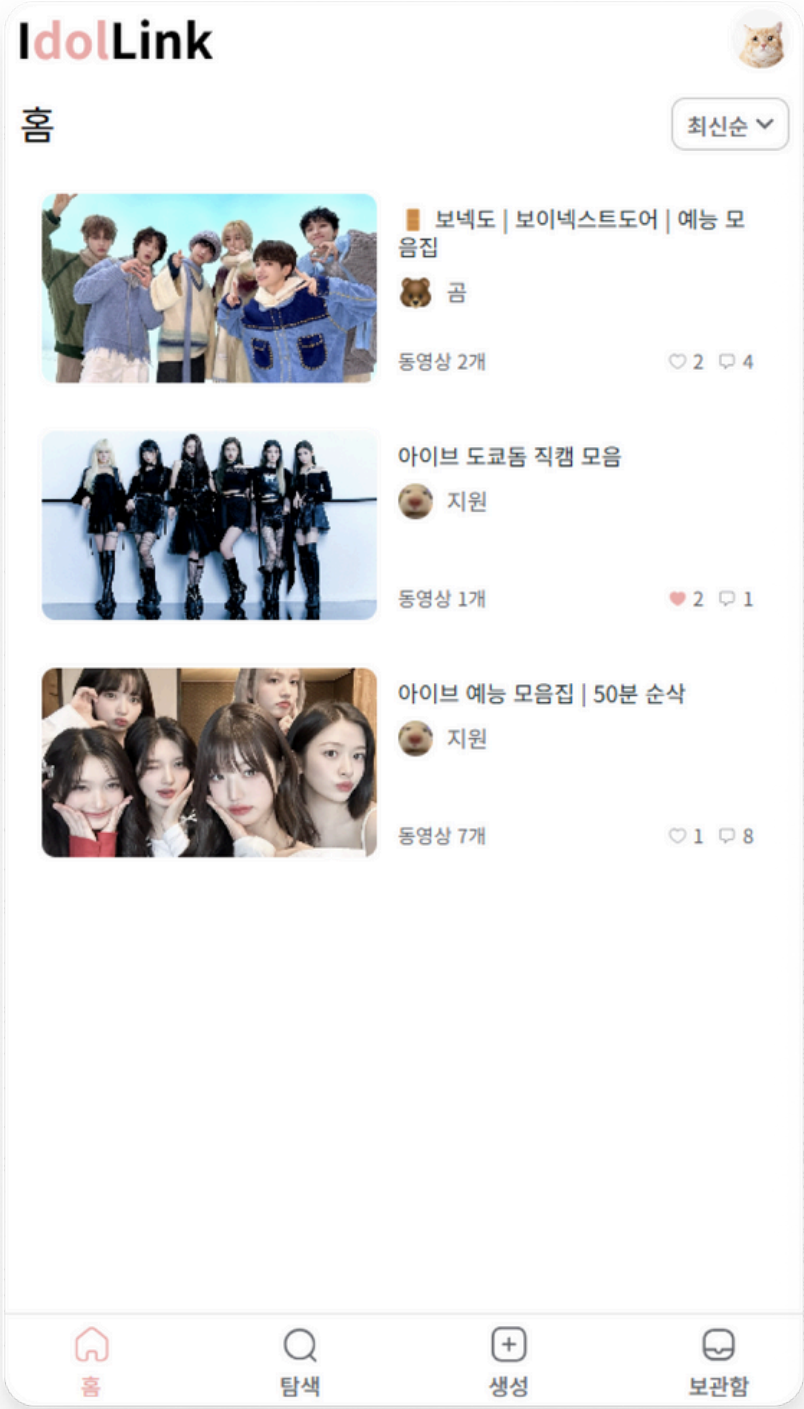
▶ 주요 작업

- 무한스크롤 구현
- 정렬 기능 구현

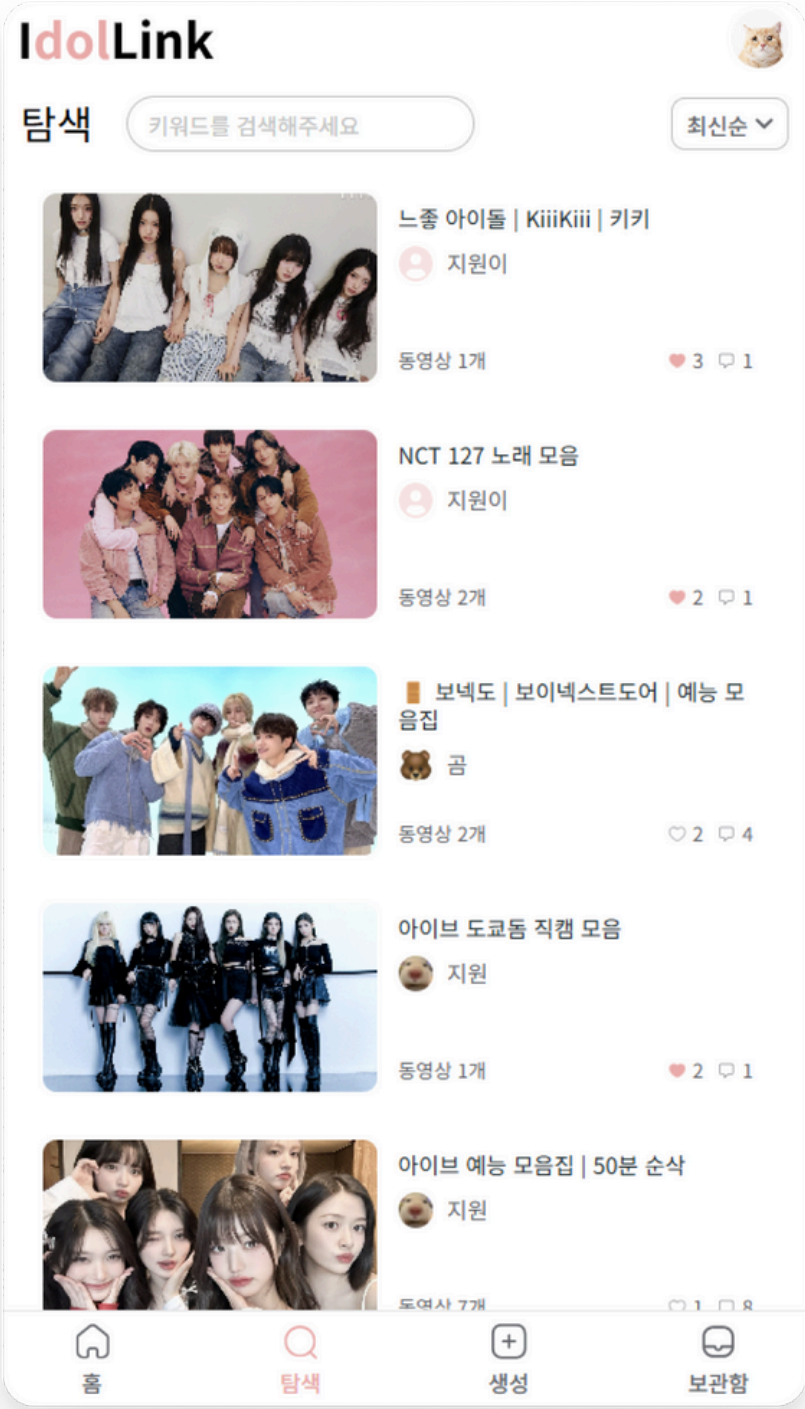
▶ 고민 지점

- 홈/탐색 페이지에서 많은 데이터를 효율적으로 로드하기 위해 초기에는 스크롤 이벤트 기반의 무한 스크롤을 구현했습니다. 그러나 스크롤할 때마다 위치 계산 이벤트가 발생하여 불필요한 연산이 반복되는 문제가 있었습니다.

이를 개선하기 위해 IntersectionObserver API로 전환했습니다. 스크롤 이벤트 방식에서는 페이지 스크롤 시 수백 회의 이벤트 핸들러가 실행되었지만, IntersectionObserver는 마지막 요소가 뷰포트에 진입하는 시점만 감지하여 1회만 호출됩니다. 추가로 useRef로 Observer 인스턴스를 관리하여 컴포넌트 리렌더링 시에도 동일한 인스턴스를 재사용하도록 최적화했습니다.



< 홈페이지 >



< 탐색 페이지 >

동영상 공유_IdolLink 프로젝트 (25.03~25.04)

[동영상 재생/댓글 페이지]

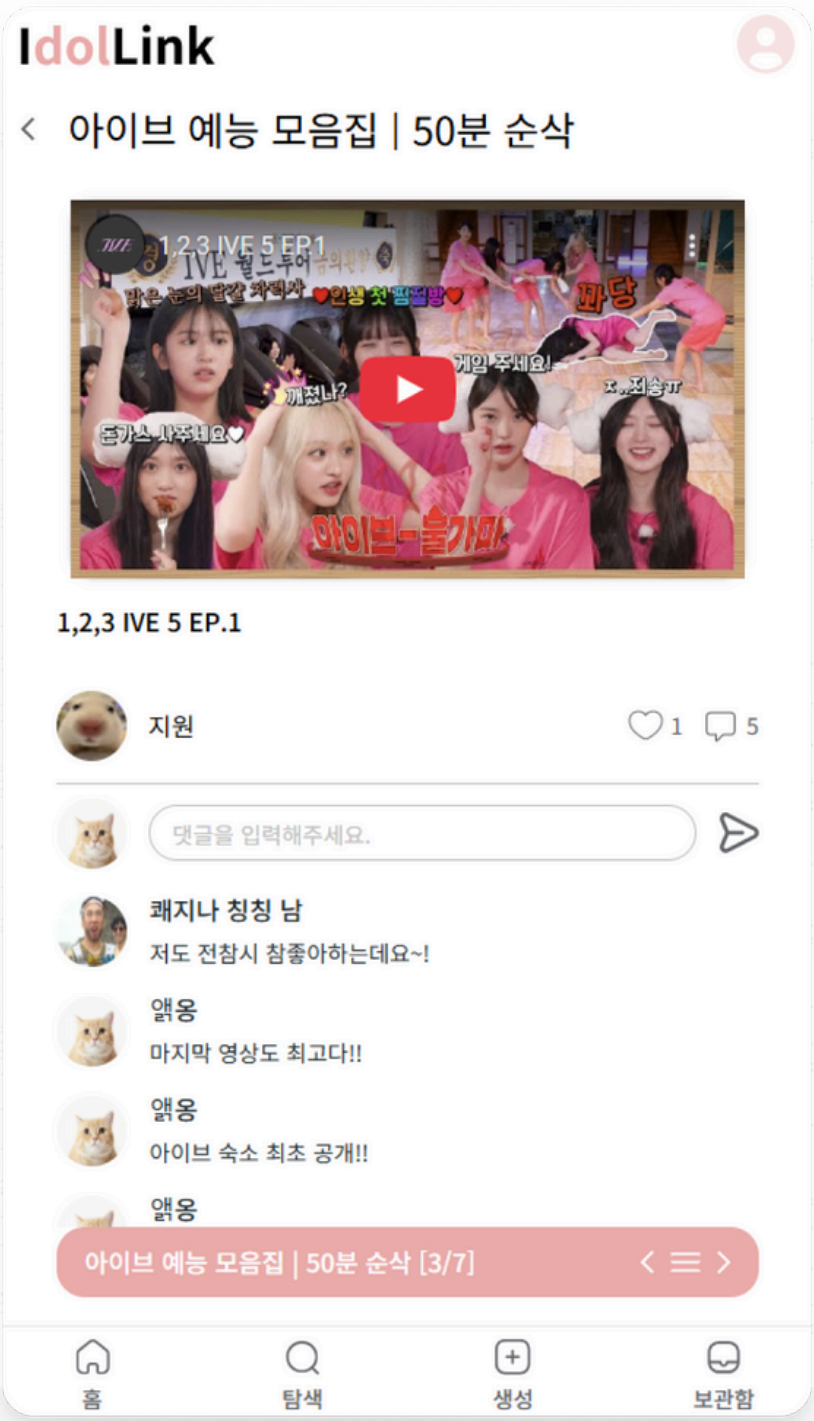
▶ 주요 작업

- 낙관적 업데이트(Optimistic UI) 활용

▶ 고민 지점

- 인스타그램, 트위터 등 주요 서비스에서 좋아요 클릭 시 즉시 UI가 반응하는 것을 보고, 동일한 UX를 제공하기 위해 낙관적 업데이트를 도입하기로 했습니다. 초기에는 setQueryData로 상태만 즉시 변경하는 방식을 사용했습니다.
그러나 네트워크 오류나 서버 검증 실패 시 UI는 이미 '좋아요 완료' 상태로 표시되었지만 실제 서버에는 반영되지 않아 데이터 불일치가 발생하는 문제를 확인했습니다. 이는 실패 시 이전 상태로 롤백하는 로직이 없었기 때문이었습니다.

React Query의 onMutate → onError → onSettled 구조를 활용해 해결했습니다. onMutate에서 mutation 실행 전 현재 상태를 context에 저장하고 즉시 UI를 업데이트한 뒤, onError에서 저장해둔 이전 상태로 자동 복원하도록 구현했습니다. 최종적으로 onSettled에서 invalidateQueries를 호출하여 성공/실패 여부와 관계없이 서버 데이터와 동기화되도록 설계했습니다.



< 재생 페이지 >